# Trust Systems for Autonomous Agent Composition:
Theory, Formalization, and Deployment

Robin Langer Claude

July 2026

**Abstract**

We describe a trust system for autonomous research agents that grades mathematical claims, validates citations, and composes across agent boundaries via email. Three agents—Clio (algebraic combinatorics), Rick (combinatorial Hopf algebras), and MacBeth (category theory)—each run independent trust systems inside Docker containers with different trust chains, different boundaries, and different evidence duties. A fourth deployment—the Warnaar loop experiment—applies the same machinery to a RAG-based research workflow where citations must survive chunk retrieval rather than dream consolidation. A monotone interface map $\varphi$ translates grades between systems; a new grade `peer-claimed` mediates cross-agent citation via email with proof attachments. The core theory (the certificate lemma, trust transport, consolidation preservation) is Lean-verified with no `sorry`. The registry structure is a directed container whose positions separate into a proof tree (`role: "premise"`) and an exploration tree (`role: "attempt"`). We describe the complete system: definitions, theorems, Lean formalization, the generic validator, and the operational deployment across four agents.

## Contents

**1 Introduction** **2**

**2 The data of a trust system** **2**

**3 The certificate lemma** **3**

**4 Morphisms and cross-agent citation** **4**
4.1 The interface maps . . . . . 4
4.2 Source-extraction demotion . . . . . 5
4.3 The validator checks . . . . . 5

**5 Consolidation preservation** **5**
5.1 The certificate gap . . . . . 5

**6 The `peer-claimed` grade** **6**
6.1 The email protocol . . . . . 6
6.2 First live composition event . . . . . 6

**7 The tree pair: premises and attempts** **6**
7.1 Directed container structure . . . . . 7
7.2 The tree pair and the directed container . . . . . 7

**8 Evidence duties** **7**

**9 The generic validator** **8**

**10 Deployment** **8**
10.1 Clio's registries . . . . . 8
10.2 Rick's registry . . . . . 9

**11 Directed container operations** **9**

**12 Lean formalization summary** **10**

**13 Three-agent citation chain** **10**

**14 The loop experiment: RAG-based citation provenance** **11**
14.1 Extraction chain . . . . . 11
14.2 Source format . . . . . 11
14.3 Trust chain . . . . . 11
14.4 Standalone deployment . . . . . 11

**15 Conclusion** **12**

# 1 Introduction

An autonomous research agent proves theorems, reads papers, and consolidates memory across cycles. Each of these operations can corrupt knowledge: a sub-agent hallucinates paper contents, compression drops citations, a proof absorbs hearsay as fact. The trust system exists to make such corruption mechanically visible.

This paper describes the complete system as deployed across four agents. It is simultaneously a mathematical paper (definitions, theorems, proofs), a systems paper (the validator, the descriptors, the email protocol), and a field report (what happened when the agents ran). The three contributions are:

1. **The theory.** Trust systems $(\mathcal{D}, T, s)$ with the certificate lemma (Lemma 3.2), trust transport (Lemma 4.2), and consolidation preservation (Theorem 5.2). All Lean-verified.

2. **The composition mechanism.** Cross-agent citation via a monotone interface map $\varphi$, mediated by email with proof attachments, graded at `peer-claimed`. The first live composition event: Rick peer-reviews Clio's claims; Clio upgrades nodes to `peer-reviewed` with the review artifact.

3. **The tree pair.** Every child node carries a `role` field (`"premise"` or `"attempt"`) that separates the downward proof tree from the outward exploration tree. The boundary rule checks only premises. This makes the directed container structure explicit: the proof tree is the successful-path coKleisli morphism; the exploration tree is the dead-ends morphism.

# 2 The data of a trust system

**Definition 2.1** (Trust system)**.** A *trust system* is a triple $(\mathcal{D}, T, s)$:

1. **Dependency structure** $\mathcal{D}$: a multicategory. Objects are *claims*. A multimorphism $f : y_1, \ldots, y_n \to x$ is a *checking step*: a derivation of $x$ from premises $y_1, \ldots, y_n$. The 0-ary case $f : () \to x$ is a *ground check* (reading a source, a Lean compilation, a reproduced computation).

2. **Check-poset** $T$: a bounded chain (linear order with $\bot$), ordered by strength of check. Meets and joins are min and max.

3. **Grading** $s : \mathrm{Mor}(\mathcal{D}) \to T$, assigning to each checking step the strength of the check it received.

$T$ is the *deployment parameter*. Each agent instantiates it differently:

| Agent | Chain $T$ | Boundary |
|---|---|---|
| Clio | `speculative` $<$ `computed` $<$ `peer-claimed` $<$ `proved` $<$ `peer-reviewed` $<$ `published` $<$ `lean-verified` | `proved` |
| Rick | `hunch` $<$ `sketched` $<$ `computed` $<$ `peer-claimed` $<$ `checked-sober` $<$ `proved` $<$ `lean-verified` | `checked-sober` |
| MacBeth | (same as Clio) | `proved` |

**Remark 2.2** (Non-isomorphic chains)**.** Rick's chain has seven levels; Clio's and MacBeth's have seven (with `peer-claimed` added between `computed` and `proved`). Rick's chain includes `hunch`, `sketched`, and `checked-sober`, which have no direct analogues in Clio's system. The interface map $\varphi$ (Definition 4.1) translates between them.

**Definition 2.3** (Grade of a derivation, canonical trust)**.** The grade of a derivation $\delta = (f; \delta_1, \ldots, \delta_n)$ is the weakest link:

$$g(\delta) = s(f) \wedge \bigwedge_i g(\delta_i).$$

The *canonical trust* of a claim $x$ is the best derivation:

$$\tau(x) = \bigvee \{g(\delta) : \delta \text{ a derivation of } x\}, \quad \text{with } \bigvee \emptyset = \bot.$$

This is a provenance-semiring instance (Green–Karvounarakis–Tannen, PODS 2007): the min-max specialization. The semiring is not the contribution; the deployment is.

**Standing hypotheses.** $\mathcal{D}$ is finite and well-founded: every claim's justification bottoms out in ground checks. Finiteness makes all sups attained; well-foundedness is load-bearing (Remark 3.3).

## 3 The certificate lemma

**Definition 3.1** (Sound assignment)**.** An assignment $t : \mathrm{Ob}(\mathcal{D}) \to T$ is *sound* if for every claim $x$ with $t(x) > \bot$ there exists a checking step $f : y_1, \ldots, y_n \to x$ with

$$t(x) \leq s(f) \wedge \bigwedge_i t(y_i). \tag{†}$$

This is the **boundary rule** as deployed: a registry node may claim level $\ell$ only if the step-check reached $\ell$ and every premise child is at least $\ell$. Condition (†) is node-local—checkable without seeing the whole tree. The validator is a certificate checker: it verifies (†) at each node; it never computes $\tau$.

**Lemma 3.2** (Certificate lemma)**.** *Under the standing hypotheses:*

*(i) $\tau$ is sound.*

*(ii) Every sound $t$ satisfies $t \leq \tau$ pointwise.*

*Hence $\tau$ is the greatest sound assignment.*

*Proof.* (i) Suppose $\tau(x) > \bot$. Since $T$ is a finite chain, the sup is attained: $\tau(x) = g(\delta)$ for some $\delta = (f; \delta_1, \ldots, \delta_n)$. Then $\tau(x) = s(f) \wedge \bigwedge_i g(\delta_i) \leq s(f) \wedge \bigwedge_i \tau(y_i)$, so (†) holds at $x$.

(ii) By well-founded induction. Let $t$ be sound and pick $f$ as in (†). By induction $t(y_i) \leq \tau(y_i)$; choose $\delta_i$ with $g(\delta_i) = \tau(y_i)$. Then $\delta = (f; \delta_1, \ldots, \delta_n)$ has $g(\delta) \geq t(x)$, so $\tau(x) \geq t(x)$.

*Lean:* `sound_le_tau`, `tau_sound` in `Trust.lean`, no sorry. □

**Remark 3.3** (Well-foundedness is load-bearing)**.** Drop well-foundedness and (ii) fails: two claims citing each other, both graded $\top$, have $t \equiv \top$ sound but $\tau \equiv \bot$. Circular citation is trust inflation that passes every node-local check. The validator checks acyclicity explicitly via DFS across agent namespaces.

*Lean:* `cyc_sound_top`, `cyc_no_deriv`, `cyc_not_wf`.

# 4 Morphisms and cross-agent citation

**Definition 4.1** (Morphism of trust systems)**.** A *morphism* $(F, \varphi) : (\mathcal{D}, T, s) \to (\mathcal{D}', T', s')$ is a multifunctor $F : \mathcal{D} \to \mathcal{D}'$ together with a monotone map $\varphi : T \to T'$ that is **strict at bottom** ($\varphi(\bot) = \bot'$) and laxly compatible with grading:

$$\varphi(s(f)) \leq s'(Ff) \quad \text{for every checking step } f.$$

Strictness at $\bot$—"a morphism maps *unchecked* to *unchecked*"—was found missing by the Lean formalization: without it, transport fails on a claim with no derivation.

**Lemma 4.2** (Transport of trust)**.** *For any morphism $(F, \varphi)$ and claim $x$:*

$$\varphi(\tau(x)) \leq \tau'(Fx).$$

*Proof.* Induction on derivations. For $\delta = (f; \delta_1, \ldots, \delta_n)$, $F\delta$ is a derivation of $Fx$ with $g'(F\delta) \geq \varphi(g(\delta))$ by monotonicity and lax compatibility. Taking sups and using attainment (finite chain) or strictness at $\bot$ (empty derivation set).

*Lean:* `tau_transport` in `Trust.lean`, no sorry. Both case splits found by the formalization. □

**Corollary 4.3** (Cross-agent citation)**.** *Clio citing a Rick registry node is composition of morphisms: Rick's system → opaque restriction at his registry roots → Clio's system along $\varphi : T_{Rick} \to T_{Clio}$. Lemma 4.2 composed twice gives the safe import grade.*

## 4.1 The interface maps

The $\varphi$ maps are declared in JSON descriptors. The critical rows:

| Direction | Key mapping | Effect |
|---|---|---|
| Rick → Clio | `checked-sober` ↦ `peer-claimed` | Below Clio's boundary |
| Rick → Clio | `proved` ↦ `proved` | At Clio's boundary |
| Clio → Rick | `peer-claimed` ↦ `peer-claimed` | Below Rick's boundary |
| Clio ↔ MacBeth | identity on all grades | Same chain |

Rick's `checked-sober` maps to `peer-claimed` (not `computed`) in Clio's system. This is forced by monotonicity: `peer-claimed` sits between `computed` and `proved` in both chains, and the mapping must preserve order.

### 4.2 Source-extraction demotion

Each interface also declares a `source_phi` that demotes extraction levels by one step: deep-read by proxy becomes abstract-level check locally. A sub-agent's summary of a paper that Rick read is two degrees of separation from the mathematics.

### 4.3 The validator checks

The generic validator (`trustcheck.py`) enforces:

1. $\varphi$ **side conditions**: total, monotone, strict at $\bot$.
2. **Trust cap**: a shared stub's local trust $\leq \varphi$(canonical trust).
3. **Artifact transport**: if $\varphi$(canonical trust) $\geq$ boundary and the canonical node has a proof file, the file must exist under the peer's artifact directory.
4. **Acyclicity**: DFS across agent namespaces detects circular citation.

## 5 Consolidation preservation

**Definition 5.1** (Consolidation)**.** A *consolidation* $C : (\mathcal{D}, T, s) \to (\mathcal{D}', T, s')$ over the same check-chain consists of:

- **Shapes forward**: a map on claims $x \mapsto Cx$ and on checking steps $f \mapsto Cf$ with premises corresponding and grades not dropping: $s(f) \leq s'(Cf)$.
- **Positions backward**: for every claim $x$ and every checking step $f' : y'_1, \ldots, y'_m \to Cx$ of the target, a chosen step $\rho(f')$ of the source with premises corresponding and the target grade cached: $s'(f') \leq s(\rho(f'))$.

$\rho$ is the position map; a citation is precisely its cached value at one step. The definition never mentions $\tau$: it is a bisimulation-style, node-local condition.

**Theorem 5.2** (Consolidation preservation)**.** *If $C$ is a consolidation and both premise relations are well-founded, then $\tau'(Cx) = \tau(x)$ for every source claim $x$.*

*Proof.* Forward: Lemma 4.2 at $\varphi = \mathrm{id}$. Backward: well-founded induction on the source, using attainment, the premise correspondence, and the cached-grade condition.

*Lean:* `tau_consolidate`, no sorry. □

**Theorem 5.3** (Consolidations compose)**.** *Consolidations have identities and compose: shapes forward through both factors, positions backward (contravariantly), caches by transitivity.*

Lean: *`Consolidation.id`, `Consolidation.comp`, `tau_consolidate_comp`, no sorry.*

### 5.1 The certificate gap

The measured "degradation" in the loop experiment was not a drop in $\tau$ but a *certificate-format gap*: $t < \tau$ where $t$ is the sound assignment the validator computes (Lemma 3.2(ii)) and $\tau$ is the canonical trust. Prose provenance is one compression step from gone: the consolidator that reads only machine certificates treats unparseable $\rho$ as absent $\rho$.

# 6 The `peer-claimed` grade

Cross-agent composition is triggered by email. When Rick proves a theorem, he emails Clio with the proof file attached. Clio registers the claim at `peer-claimed`—a grade between `computed` and `proved` that means: "a trusted peer says they proved this, and I have their artifact, but I haven't independently verified it."

| Field | Value | Purpose |
|---|---|---|
| `trust` | `peer-claimed` | Below boundary |
| `claimed_by` | `"rick -- peers/rick/emails/2026-07-05.md"` | Evidence duty (required) |
| `file` | `"peers/rick/proofs/bn-mechanism.md"` | The proof artifact |
| `role` | `"premise"` | Part of the proof tree |

`peer-claimed` sits below the boundary in every chain:

- Clio: below `proved`. Upgrade: read the proof → `proved`.
- Rick: below `checked-sober`. Upgrade: re-derive it sober → `checked-sober` (with the `recheck` field).

The asymmetry is the composition being honest about what "verification" means in each system.

## 6.1 The email protocol

1. Agent proves theorem, runs validator, proof file exists.
2. Agent emails peers with proof file attached (`-attachments`).
3. Recipient saves email artifact and attachment to `peers/<agent>/`.
4. Recipient adds registry node at `peer-claimed` with `claimed_by`.
5. Recipient reads proof, upgrades if satisfied.

Peer review works the same way in reverse: if the reviewer endorses, the node upgrades to `peer-reviewed`. If the reviewer finds a gap, the node demotes (the objection becomes the `reason` if it kills the route).

## 6.2 First live composition event

On 2026-07-05, Rick reviewed two of Clio's claims (c=5 interior theorem and the anchor-content formula). He independently reproduced $f^{(13,13,6)} = 230{,}814{,}869{,}760$ and verified $\beta'(5) = 3$. Clio saved the review artifact at `proofs/reviews/2026-07-05-rick-review-clio.md` and upgraded both nodes from `proved` to `peer-reviewed`. The registry event mechanism fired on a real email, produced a real upgrade, with a real artifact.

# 7 The tree pair: premises and attempts

Each node in a registry tree has children, but children serve two different roles:

**Definition 7.1** (Tree pair)**.** Every non-root child node carries a `role` field:

- `"premise"`: this child is a dependency. The parent's proof stands on it. The boundary rule checks premises.

- `"attempt"`: this child is an approach that was tried. Dead ends are always attempts. The boundary rule skips attempts.

This separates two trees:

- The **downward proof tree** (premise children): what the theorem depends on. Filtering to role = premise gives the proof skeleton—the `successful-path` report.
- The **outward exploration tree** (attempt children): what was tried and abandoned. Filtering to role = attempt gives the search history—the `dead-ends` report.

### 7.1 Directed container structure

The registry is a directed container (Ahman–Chapman–Uustalu). Shapes are search trees, positions are paths:

- root: the conjecture (the empty path).
- sub$(t, p)$: the sub-search at path $p$.
- shift$(t, p, q)$: path composition—$q$ inside the sub-search at $p$, seen globally.

The induced comonad: extract reads the root's trust; duplicate replaces every node with the full sub-search below it. The reports are coKleisli morphisms $W(\text{Registry}) \to \text{Report}$.

*Lean:* `ProofSearchN.lean` formalizes rose trees with arbitrary finite branching: `sub`, `shift`, `shift_root` (D3), `sub_shift` (D4), `shift_assoc` (D5), all no sorry. `TrustBoundary.lean` proves the boundary rule is stable under duplicate (`ok_sub`)—validating the whole tree validates every subtree.

### 7.2 The tree pair and the directed container

The `role` field makes the two coKleisli projections explicit in the data:

- `successful-path`: filter to role = premise, trust $\geq$ boundary.
- `dead-ends`: filter to role = attempt, trust = dead-end.
- `frontier`: role = premise, trust $<$ boundary, not dead-end.

Without the role field, the boundary rule conflated premises and attempts: a `proved` node with a `computed` attempt child failed the boundary check, even though the attempt was exploration, not a dependency. With the role field, the boundary rule checks only premises.

## 8 Evidence duties

Certain trust levels require evidence fields. These are declared in the deployment descriptor and enforced by the validator:

| Level | Field | Severity | Notes |
|---|---|---|---|
| `peer-claimed` | `claimed_by` | error | All agents |
| `checked-sober` | `recheck` | error | Rick only |
| `proved` | `file` | error | Rick only (artifact) |
| `peer-reviewed` | `review` | error | Clio, MacBeth |
| `published` | `publication` | error | Clio, MacBeth |
| `lean-verified` | `lean` | warning | All agents |

Rick's evidence duties are stricter than Clio's: `checked-sober` requires the recheck date and path, `proved` requires the proof file. Rick promotes earlier (lower boundary) and documents harder.

# 9 The generic validator

`trustcheck.py` is a single Python script (stdlib only) parametrized over a deployment descriptor—the $(T, \varphi)$ data as JSON. It checks:

1. **Well-formedness**: tree structure, required fields (`id`, `approach`, `trust`, `children`, `role`), valid trust values, unique ids.
2. **Boundary rule**: for nodes at or above boundary, every *premise* child must be at or above boundary. Attempt children are skipped.
3. **Evidence duties**: per-level field requirements from the descriptor.
4. **Sources**: extraction-level blocking (a `proved` claim cannot rest on an `agent-summary` source).
5. **Cross-agent**: $\varphi$ lint (total, monotone, strict at $\bot$), shared-stub trust cap, artifact transport, acyclicity DFS.
6. **Reports**: `successful-path`, `dead-ends`, `frontier`, `cross-refs`, `certificate-gap`, `shallow`, `footprint`.

The validator is a certificate checker for the sound assignment of Lemma 3.2. It verifies (†) at each node; it never computes $\tau$.

**Regression:** the generic validator produces byte-identical verdicts with the three legacy per-agent validators on all deployed registries and source indexes.

# 10 Deployment

Four agents are deployed:

| Agent | Container | Chain | Status |
|---|---|---|---|
| Clio | Docker on OCI | 7 levels, boundary at `proved` | Running |
| Rick | Docker on OCI | 7 levels, boundary at `checked-sober` | Running |
| MacBeth | Docker on OCI | 7 levels, boundary at `proved` | Stopped |
| Loop | Local workflow | 5 levels, boundary at `proved` | Running |
| Template | `RaggedR/dreaming-agent` | 7 levels, boundary at `proved` | Template |

Each agent has:

- The validator and all peer descriptors in `code/`.
- `peers/<agent>/{emails, proofs, registry}/` directories.
- Prompts instructing email with attachments for proved results.
- Prompts instructing `peer-claimed` registration for received emails.
- Backup cron (12-hourly) to GitHub, covering prompts + projects.

## 10.1 Clio's registries

Clio has two active registries:

- **Three-row $|J^*|$-even** (17 nodes): 5 `lean-verified`, 2 `peer-reviewed`, 7 `proved`, 3 `dead-end`. Active frontier at general-$c$.
- **Staircase T-system** (3 nodes): 2 `dead-end` with `computed` refutations carrying structural reasons.

### 10.2 Rick's registry

Rick created his first registry on 2026-07-05 (strict #AXIS closed form, 7 nodes): 1 `lean-verified`, 2 `proved`, 2 `computed`, 1 `sketched`, 1 `dead-end`. His evidence duties (`recheck`, `file`) fired correctly on first contact.

## 11 Directed container operations

The directed container operations are exposed via `trustcheck.py ops`, a subcommand that provides both the core algebra (pure tree operations independent of any deployment) and deployment-parametric operations that use the trust chain. Positions are index-tuples: `()` is the root, `(2,)` is the third child, `(1, 0)` is the first child of the second child. `shift` is tuple concatenation (the free monoid on child indices).

| Operation | Mathematical role | Lean | Python | CLI |
|---|---|---|---|---|
| `extract` | Counit $\varepsilon : W(A) \to A$ | `counit` | `dc_extract` | `ops` |
| `sub` | sub : $S \to P(S) \to S$ | D1–D5 | `dc_sub` | `ops` |
| `shift` | Path concatenation $p +\!\!+ q$ | D3–D5 | `dc_shift` | (libr |
| `duplicate` | Comultiplication $\delta$ | `comult` | `dc_duplicate` | `ops` |
| `extend` | CoKleisli extension | `extend` | `dc_extend` | `ops` |
| `compose` | $g .* f = g \circ W(f) \circ \delta$ | `cokleisli_comp` | `dc_compose` | (libr |
| `strength` | $\sigma : W(A \times B) \to A \times W(B)$ | `strength` | `dc_strength` | `ops` |
| `tau` | Canonical trust $\tau$ (greatest sound) | `tau_sound` | `dc_tau` | `ops` |
| `resolve-transitive` | Multi-hop $\varphi$ composition | — | `resolve_transitive` | `ops` |

`shift` and `compose` are Python library functions rather than CLI subcommands: `shift` is tuple concatenation (one line), and `compose` requires passing two Python functions (not expressible on a command line). Both are called internally by the other operations.

**CoKleisli composition.** Given coKleisli morphisms $f : W(X) \to Y$ and $g : W(Y) \to Z$, their composite $g.{*}f = g \circ \text{extend}(f)$ is again a coKleisli morphism. Associativity (`cokleisli_assoc` in `CoKleisli.lean`) follows from `extend_extend`, which itself follows from coassociativity of the comonad. The right unit law $\text{extend}(\varepsilon) = \text{id}$ (`extend_counit`) is verified by `ops extend`: applying `dc_extract` at every position recovers the original labels.

**Strength.** The canonical strength $\sigma\langle s, v\rangle = (\pi_1(v(o(s))),\ \langle s, \pi_2 \circ v\rangle)$ factors a uniform context (conjecture, status, dates) out of the labelled tree. Naturality in both variables is proved in `CoKleisli.lean`.

**Canonical trust ($\tau$).** `ops tau` computes canonical trust bottom-up: $\tau(n) = \min(\text{own_rank},\ \min_i \tau(c_i))$ over premise children (attempt children excluded, matching the boundary rule). Each node is classified as TIGHT ($\tau$ = claimed), CONSERVATIVE ($\tau >$ claimed, safe to upgrade), or INFLATED ($\tau <$ claimed, boundary violation). This closes the Lean–Python gap: the certificate lemma (Lemma 3.2) says $\tau$ is the greatest sound assignment; the Python validator checks soundness (no inflation) and now also computes $\tau$ (optimality).

**Multi-hop morphism composition.** `ops resolve-transitive` follows shared stubs across registries, composing $\varphi$ at each hop with cycle detection. When Clio cites Rick citing MacBeth, the composed $\varphi = \varphi_{\text{Clio}\leftarrow\text{Rick}} \circ \varphi_{\text{Rick}\leftarrow\text{MacBeth}}$ is computed and applied. Monotonicity of the composite follows from monotonicity of each factor (Lemma 4.2).

## 12 Lean formalization summary

| File | Content | Sorry |
|---|---|---|
| `Basic.lean` | Containers, extensions, representation theorem (Yoneda) | 0 |
| `Directed.lean` | Directed containers (D1–D5), all three comonad laws | 0 |
| `DContCat.lean` | Directed containers as categories, cofunctors, composition | 0 |
| `ProofSearchN.lean` | N-ary rose trees as a directed container | 0 |
| `Trust.lean` | Trust systems, certificate lemma, transport, consolidation preservation, composition | 0 |
| `TrustBoundary.lean` | Boundary-rule stability under duplicate, deployed chain | 0 |
| `CoKleisli.lean` | CoKleisli extension, composition, strength | 0 |
| `ProofSearch.lean` | Binary proof search trees | 0 |
| `ZappaSzep.lean` | Zappa-Szép products of directed containers | 0 |

The canonical location is `macbeth-seed/lean/Containers/Containers/` (a buildable Lean 4 project); `trust-boundaries/lean/` holds byte-identical reference copies of all nine files.

Key declarations: `sound_le_tau`, `tau_sound`, `tau_transport`, `tau_consolidate`, `tau_consolidate_comp`, `cyc_sound_top`/`cyc_no_deriv`/`cyc_not_wf`, `ground_tau_hi`/`layer4_tau_bot` (degradation counterexample), `ok_sub` (boundary stability), `left_counit`/`right_counit`/`coassoc` (comonad laws), `extend_extend` (coKleisli associativity), `strength_natural_left`/`strength_natural_right` (strength naturality).

The formalization corrected the prose in three places: (1) $T$ must be a chain for attainment; (2) morphisms must be strict at $\bot$; (3) two case splits in the proof of Lemma 4.2.

## 13 Three-agent citation chain

The first end-to-end test of multi-agent trust composition exercises the full chain MacBeth → Rick → Clio, with each hop crossing an agent boundary through a different interface map.

| Agent | Registry | Trust | Cites | Via |
|---|---|---|---|---|
| MacBeth | comonad-laws | `lean-verified` | — | — |
| Rick | boundary-stability | `proved` | MacBeth | `shared: macbeth/comonad-laws#root` |
| Clio | trust-composition | `proved` | Rick | `shared: rick/boundary-stability#root` |

The composed $\varphi$ maps MacBeth's grades through Rick's non-isomorphic chain (which has `hunch`, `sketched`, and `checked-sober` — levels absent from both Clio's and MacBeth's chains) and into Clio's:

$$\varphi_{\text{Clio}\leftarrow\text{Rick}} \circ \varphi_{\text{Rick}\leftarrow\text{MacBeth}} : T_{\text{MacBeth}} \to T_{\text{Clio}}$$

$$\texttt{lean-verified} \mapsto \texttt{lean-verified} \mapsto \texttt{lean-verified}$$

$$\texttt{peer-reviewed} \mapsto \texttt{proved} \mapsto \texttt{proved}$$

$$\texttt{speculative} \mapsto \texttt{hunch} \mapsto \texttt{speculative}$$

Monotonicity is preserved through both hops (Lemma 4.2 composes).

The test is self-referential: the registries are *about* the correctness of the trust system. MacBeth proves the directed container structure; Rick uses it to prove boundary-rule stability under duplicate; Clio combines that with $\tau$-transport to prove cross-agent composition works. The subject matter of the registries is the correctness of the registries — not circular (no shared refs point back up the chain), but the system validating its own mathematics. The test fixture is committed at `trustcheck/tests/three-agent-chain/`.

# 14 The loop experiment: RAG-based citation provenance

The three Docker agents (Clio, Rick, MacBeth) face a common threat: *dream consolidation* compresses notes across cycles, eating citations. The loop experiment—a local research workflow for Warnaar's $A_2$ Andrews–Gordon positivity conjecture—faces a structurally different threat: *RAG retrieval*.

The knowledge pipeline is:

$$\text{RAG query} \to \text{chunk retrieval} \to \text{scratch files} \to \text{synthesis layers} \to \text{proof } \texttt{.tex}.$$

Each stage compresses. The characteristic failure mode is **chunk windowing**: a retrieved chunk is a window into a paper. The theorem's hypotheses, variable conventions, and normalizations may live in a chunk that was *not* retrieved. The text is real; the reading is wrong because the context is missing. This is the RAG analogue of the phantom determinant—but structurally different, because the source material is genuine, only incomplete.

## 14.1 Extraction chain

The loop uses a five-level extraction chain tuned to the RAG pipeline:

| Loop level | Meaning | Agent analogue |
|---|---|---|
| `recalled` | Training memory; no chunk | (none) |
| `rag-summary` | Only the chunk's summary tag | `agent-summary` |
| `chunk-read` | Retrieved chunk read verbatim | `abstract` |
| `context-read` | Surrounding chunks checked | `deep-read` |
| `paper-read` | Relevant section at line level | `verified-quote` |

The blocking rule: below `context-read` blocks `proved`. A fact below `context-read` may not be load-bearing—the hypotheses may live in an unretrieved chunk.

## 14.2 Source format

Sources are keyed by **paper slug** (the directory name under `literature/chunks/`), not arXiv ID. The slug resolves on disk: `literature/chunks/<slug>/chunk_NNN.tex`. The machine regex matches `paper_slug/chunk_NNN`; the on-disk check catches hallucinated chunk references outright—a cited chunk that does not exist in the corpus is an error, not a warning.

## 14.3 Trust chain

The loop's trust chain is shorter than the agents':

$$\texttt{speculative} < \texttt{computed} < \texttt{proved} < \texttt{verified} < \texttt{lean-verified},$$

with boundary at `proved`. The `verified` grade means an independent verifier agent (or Robin) hostile-reviewed the proof; it requires a `review` field. No `peer-claimed`, `peer-reviewed`, or `published`—those belong to the agents' cross-agent email protocol. No cross-agent interfaces: the loop is standalone.

## 14.4 Standalone deployment

The loop experiment has no Docker container and no cross-agent interfaces ($\varphi$ maps). Its deployment descriptor declares only the trust chain, the blocking rules, and the citation regex. The generic validator (`trustcheck.py`) applies the same certificate-checking machinery—Lemma 3.2 is parametric in $T$—but without the cross-agent checks of §4.

# 15 Conclusion

The trust system is live. Four agents (Clio, Rick, MacBeth, and the public `dreaming-agent` template) and the Warnaar loop experiment (§14) run independent trust systems with different chains, different boundaries, and different evidence duties. The Docker agents compose via email with proof attachments, graded at `peer-claimed`, with the interface map $\varphi$ translating grades between systems. The loop experiment demonstrates that the same certificate-checking machinery (Lemma 3.2) applies to a RAG-based workflow with a structurally different extraction chain and a different failure mode (chunk windowing vs. dream consolidation). The first live composition event—Rick peer-reviewing Clio's claims, Clio upgrading to `peer-reviewed` with the review artifact—occurred on the system's first day of deployment. The first three-agent citation chain (§13)—MacBeth's Lean formalization flowing through Rick's boundary-stability proof into Clio's composition theorem, with $\varphi$ translating non-isomorphic trust chains at each hop—validates the full composition machinery.

The full directed container structure is now implemented: all nine operations (§11) are deployed, the coKleisli category laws are Lean-verified (`CoKleisli.lean`), and the canonical trust $\tau$ is computed by the Python validator, closing the gap between what Lean proves (optimality) and what the validator checks (soundness). Nine Lean files, zero sorry.

The measurement that remained was: after consolidation cycles, does a citation survive in machine format, or does it decay to prose?

**It survives.** Running `certificate-gap` across all of Clio's connection files—the most heavily consolidated corpus, spanning May through July 2026 and dozens of dream cycles—yields:

| Category | Count | Meaning |
|---|---|---|
| Machine citations (valid) | 320 | arXiv IDs that resolve in `sources.json` |
| Mode 2 (prose, resolvable) | 10 | file paths on disk, not in machine format |
| Mode 1 (prose, unresolvable) | 56 | prose tokens that may be decayed citations |
| Dangling machine refs | 0 | hallucinated citations |

320 machine-format citations survived consolidation. Zero dangling. The citation discipline—arXiv ID plus locator at the point of use, with extraction levels in `sources.json`—empirically preserves the position map through the consolidation pipeline. Theorem 3 (`tau_consolidate`) predicts this: a consolidation carrying a genuine position map preserves canonical trust on the nose; the citation convention *is* the position map, implemented as a discipline rather than a data structure.

The 56 mode-1 tokens are the residue: prose references (`transfer_operators.py`, author-name shorthands) that the `certificate-gap` scanner flags as potential decayed citations. Some are genuine file references, not paper citations; a fraction may be citations that lost their arXiv ID during an early consolidation cycle before the provenance convention was installed (May 2026). The convention was introduced in July after the Gatzweiler–Krattenthaler hallucination incident (§5); citations written before it cannot retroactively carry machine format. The 10 mode-2 references (file paths that resolve on disk but are not machine citations) are backfill candidates: upgrading them to machine format is a ten-second edit per file.

**Rick's corpus: the control group.** Running the same measurement on Rick's connection and topic files—a comparable corpus (dozens of connection files, multiple topic files) but with the citation provenance discipline installed only on July 5, much later than Clio's—reveals a different pattern:

| | Clio | Rick | |
|---|---|---|---|
| Sources registered in index | 58 | 7 | |
| Machine citations (in index) | 373 | 6 | resolve in `sources.json` |
| Machine citations (unregistered) | 0 | 163 | arXiv IDs present but not indexed |
| Unique unregistered IDs | 0 | 97 | distinct papers |
| Deep-reads | multiple | 0 | all 7 entries are `agent-summary` |
| Dangling | 0 | 0 | |

The comparison isolates what the provenance discipline adds. Both agents' browse agents write arXiv IDs naturally—Rick has 97 unique IDs in machine format across his consolidated files, all written by browse agents before any provenance convention existed. *Machine-format citations survive consolidation even without the discipline.* What the discipline adds is the *index*: knowing which citations are agent-summaries versus deep-reads, tracking corrections, and—critically—catching hallucinated extractions before they become load-bearing. Rick's 97 unregistered citations are formally unchecked: any one of them could be the next Gatzweiler–Krattenthaler phantom. Clio's zero unregistered citations mean every machine-format reference in her corpus has an extraction level, a correction history, and a provenance floor.

The takeaway: citation *format* survives consolidation by default (browse agents write IDs). Citation *provenance*—the extraction level, the position map that Theorem 3 requires—must be installed. Format is necessary; provenance is sufficient.

## References


[ACU15] D. Ahman, J. Chapman, and T. Uustalu. Directed containers as categories. *EPTCS*, 207:89–98, 2015.

[GKT07] T. J. Green, G. Karvounarakis, and V. Tannen. Provenance semirings. In *PODS*, pages 31–40, 2007.

[BDGP25] N. Brinke, A. Dawar, E. Grädel, and L. Pago. Preservation theorems in semiring semantics. arXiv:2605.10829, 2025.